

Código da Função de Validação

JavaScript validador de CPF

```
// =====  
// FUNÇÃO DE VALIDAÇÃO MATEMÁTICA DE CPF  
// =====  
function validarCPF(cpf) {  
    // 1. Limpa pontuações (pontos e traço) mantendo apenas números  
    cpf = cpf.replace(/\D/g, '');  
  
    // 2. Verifica se possui 11 dígitos ou se é uma sequência repetida conhecida  
    if (cpf.length !== 11 || /^(d)\1{10}$/.test(cpf)) {  
        return false;  
    }  
  
    // 3. Cálculo do 1º Dígito Verificador (Pesos de 10 a 2)  
    let soma = 0;  
    for (let i = 0; i < 9; i++) {  
        soma += parseInt(cpf.charAt(i)) * (10 - i);  
    }  
    let resto = (soma * 10) % 11;  
    if (resto === 10 || resto === 11) resto = 0;  
    if (resto !== parseInt(cpf.charAt(9))) return false;  
  
    // 4. Cálculo do 2º Dígito Verificador (Pesos de 11 a 2)  
    soma = 0;  
    for (let i = 0; i < 10; i++) {  
        soma += parseInt(cpf.charAt(i)) * (11 - i);  
    }  
    resto = (soma * 10) % 11;  
    if (resto === 10 || resto === 11) resto = 0;  
    if (resto !== parseInt(cpf.charAt(10))) return false;  
  
    return true; // CPF Válido  
}
```

Questionário Explicativo Linha por Linha

1. **function validarCPF(cpf)**
 - **O que é?** Declaração de uma **função** que recebe como parâmetro o texto digitado no campo de CPF.
2. **cpf = cpf.replace(/\D/g, '');**
 - **O que faz?** Aplica uma **Expressão Regular (RegEx)**. O **\D** busca qualquer caractere que **não seja número** e o remove, deixando apenas os dígitos limpos.
3. **if (cpf.length !== 11 || /^(d)\1{10}\$/.test(cpf))**
 - **O que é?** Uma **Estrutura Condicional (if)** que utiliza operadores lógicos (**!== DIFERENTE**, **|| OU**).
 - **O que valida?** Testa se a quantidade de números é diferente de 11 dígitos **OU** se o CPF é uma sequência com todos os números iguais (ex: 111.111.111-11), que passa na matemática mas é um CPF inválido.
4. **return false;**
 - **O que é?** Comando de interrupção da função que retorna um **valor booleano** (false), indicando que a validação falhou.

5. **let soma = 0;**
 - **O que é?** Declaração de uma **variável mutável (let)** do tipo numérico, iniciada em 0 para acumular a soma das multiplicações dos dígitos.
6. **for (let i = 0; i < 9; i++)**
 - **O que é?** Um **Lço de Repetição (for)**. Ele executa o bloco de código 9 vezes, variando o contador i do índice 0 ao 8 (primeiros 9 dígitos do CPF).
7. **soma += parseInt(cpf.charAt(i)) * (10 - i);**
 - **O que faz?**
 - cpf.charAt(i) pega o caractere da posição atual.
 - parseInt() converte esse caractere em **número inteiro**.
 - * (10 - i) multiplica o dígito pelo seu peso decrescente (10, 9, 8... até 2).
 - += acumula o resultado dentro da variável soma.
8. **let resto = (soma * 10) % 11;**
 - **O que é o %?** É o **Operador de Módulo**, que calcula o resto da divisão inteira por 11 após multiplicar a soma por 10.
9. **if (resto === 10 || resto === 11) resto = 0;**
 - **O que faz?** Trata a regra oficial do CPF: se o resto da divisão for 10 ou 11, o dígito verificador considerado deve ser obrigatoriamente 0.
10. **if (resto !== parseInt(cpf.charAt(9))) return false;**
 - **O que faz?** Compara se o resto calculado é **diferente** do 10º dígito real digitado pelo usuário (charAt(9)). Se for diferente, o CPF é inválido e a função retorna false.
11. **for (let i = 0; i < 10; i++) / parseInt(cpf.charAt(10))**
 - **O que faz?** Repete exatamente a mesma lógica para calcular e validar o **2º dígito verificador**, agora incluindo os 10 primeiros dígitos com pesos variando de 11 a 2.
12. **return true;**
 - **O que é?** Se o código passar por todas as verificações sem cair nos return false, a função retorna o valor booleano true, confirmando que o CPF é válido.